Introduction to Deep Learning

Session 3: Deep Nets

March 2026

Magda Gregorová - magda.gregorova@thws.de

Technical University
of Applied Sciences
Würzburg-Schweinfurt

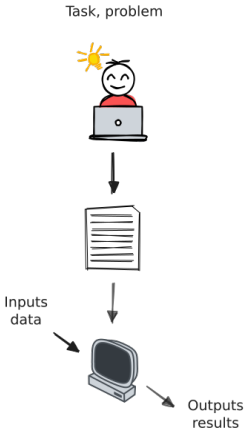
Licensed according to CC-BY-SA 4.0

Lecture Overview

1. Supervised Learning
2. Regression - Linear / Nonlinear
3. Rectified Linear Unit
4. Shallow Neural Network
5. Universal Approximation
6. Biological Inspiration

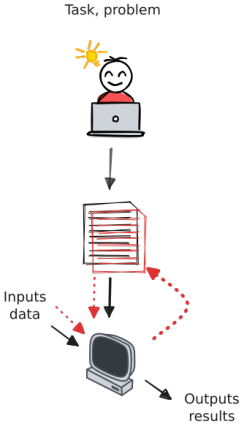
Why Machine Learning

Traditional Software



adapt to changing conditions
too many variations to capture
too complex to understand fully
too expensive to compute

Machine Learning

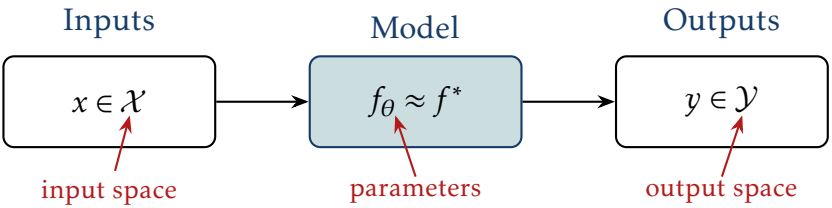


Supervised Learning

Setup: data pairs $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ with $\mathbf{x} \in \mathcal{X}$, $y \in \mathcal{Y}$

Assumption: exists true relationship $f^* : \mathcal{X} \to \mathcal{Y}$, $y = f^*(\mathbf{x})$, but f^* is never observed!

Goal: learn $f_\theta : \mathcal{X} \to \mathcal{Y}$ from $\mathcal{D}$ such that $f_\theta \approx f^*$ $\Rightarrow$ predict on **unseen** inputs $\mathbf{x} \notin \mathcal{D}$

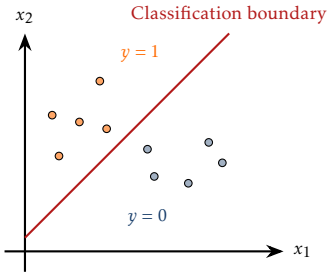


Notation: $f_\theta(\mathbf{x}) = f(\mathbf{x}; \theta) = \hat{y}$

Supervised Learning - Basic Tasks

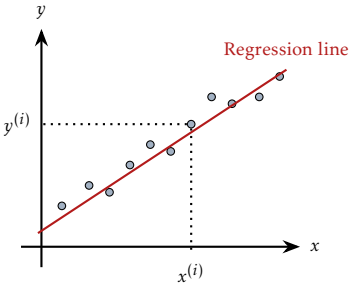
Classification (binary)

- Output: discrete label $\mathcal{Y} = \{0, 1\}$
- Goal: find $f_\theta : \mathcal{X} \to \{0, 1\}$
- Predict category for input



Regression

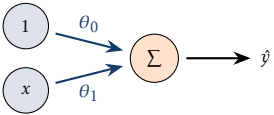
- Output: continuous value $\mathcal{Y} = \mathbb{R}$
- Goal: find $f_\theta : \mathcal{X} \to \mathbb{R}$
- Predict numerical value for input

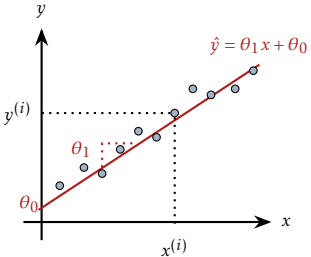


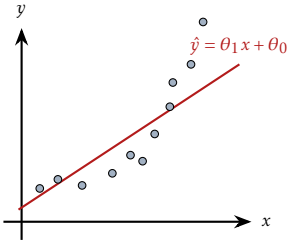
Regression and Linear Model

Simplest model: linear function

$$\hat{y} = f_\theta(x) = \theta_1 x + \theta_0, \qquad \boldsymbol{\theta} = (\theta_0, \theta_1)$$







Limitation: real data is rarely linear

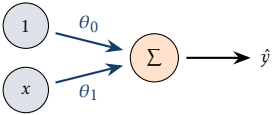
Solution: more expressive f_θ

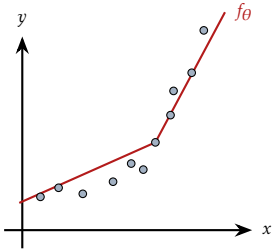
How: $f_\theta = \sin, \log, \sqrt{\ }, \exp, \ldots$ **?**

Nonlinear Regression

Idea: piecewise linear functions

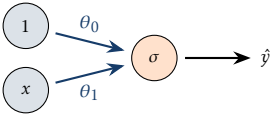
Linear: $\hat{y} = f_\theta(x) = \theta_1 x + \theta_0$

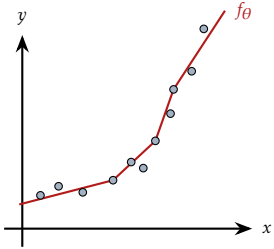




Nonlinear: $\hat{y} = f_\theta(x) = \sigma(\theta_1 x + \theta_0)$

$\sigma : \mathbb{R} \to \mathbb{R}$ - nonlinear activation function

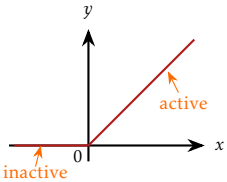




Activation function: Rectified Linear Unit (ReLU)

Rectified Linear Unit

$$\sigma(x) = \mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

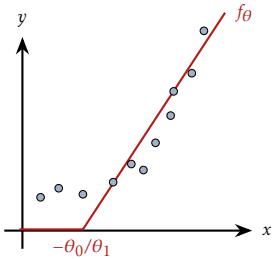


ReLU over linear: $\hat{y} = f_\theta(x) = \sigma(\theta_1 x + \theta_0)$

Effect: one bend at $x = -\theta_0/\theta_1$

Limitation: only one breakpoint - not expressive enough

Solution: combine multiple ReLUs



Combining Multiple ReLUs

Combine: weighted sum of ReLU functions

$$\hat{y} = f_\theta(x) = \theta_0 + \theta_1 h_1 + \theta_2 h_2$$

$$h_1 = \sigma(\theta_{11}\, x + \theta_{10}) \quad h_2 = \sigma(\theta_{21}\, x + \theta_{20})$$

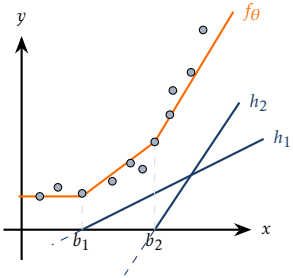
Effect: one bend per ReLU term

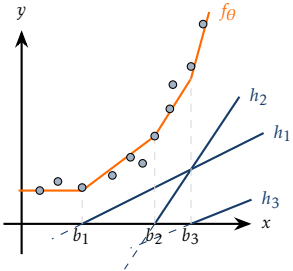
$$\hat{y} = f_\theta(x) = \theta_0 + \theta_1 h_1 + \theta_2 h_2 + \theta_3 h_3$$

$$h_3 = \sigma(\theta_{31}\, x + \theta_{30})$$

Key result: more breakpoints $\Rightarrow$ more expressive f_θ

$$\hat{y} = f_\theta(x) = \theta_0 + \sum_j \theta_j\, \sigma\big(\theta_{j1}\, x + \theta_{j0}\big)$$





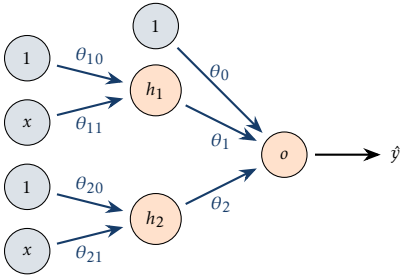
Two Linear Functions + ReLU

Mathematical representation :

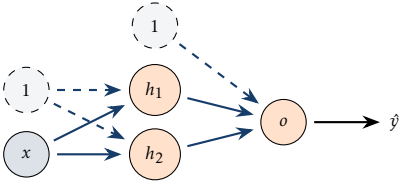
$$\hat{y} = f_\theta(x) = \theta_0 + \theta_1 h_1 + \theta_2 h_2$$

$$h_1 = \sigma(\theta_{11}\, x + \theta_{10}) \quad h_2 = \sigma(\theta_{21}\, x + \theta_{20})$$

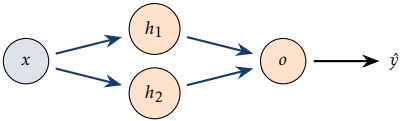
Graphical representation:



Simplified graphical representation:



Most simple graphical representation:



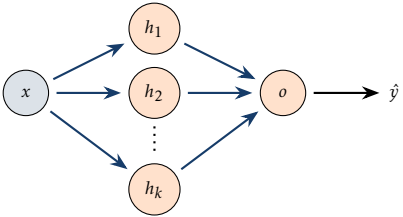
1-Hidden Layer Neural Network

Multiple neurons, single input

$$\hat{y} = f_\theta(x) = \theta_0 + \sum_j^k \theta_j h_j$$

$$h_j = \sigma(\theta_{j1} x + \theta_{j0})$$

Layers: input hidden output

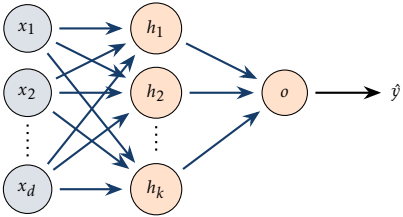


Multiple neurons, multiple inputs

$$\hat{y} = f_\theta(\mathbf{x}) = \theta_0 + \sum_j^k \theta_j h_j$$

$$h_j = \sigma(\sum_i^d \theta_{ji} x_i + \theta_{j0})$$

Layers: input hidden output



Universal Approximation

thws Technical University of Applied Sciences Würzburg-Schweinfurt

Universal Approximation Theorem

For any continuous function $g : \mathcal{X} \to \mathbb{R}$ and any $\varepsilon > 0$, there exists a 1-hidden-layer neural network f_θ such that:

$$\sup_{\mathbf{x} \in \mathcal{X}} |f_\theta(\mathbf{x}) - g(\mathbf{x})| < \varepsilon$$

What it tells us

- MLPs are universal approximators
- Enough neurons can fit any continuous function

What it doesn't tell us

- How many neurons are needed
- How to find the parameters $\boldsymbol{\theta}$
- Whether the model will generalize

Binary Classification

Linear classifier:

$$\hat{y} = \mathbb{1}\left[\theta_0 + \sum_i \theta_i x_i \geq 0\right]$$

Decision boundary:

Hyperplane $\theta_0 + \boldsymbol{\theta}^\top \mathbf{x} = 0$

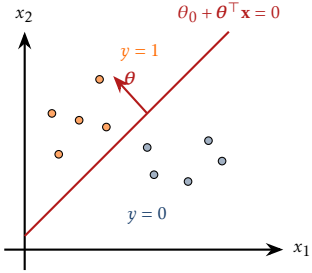
Limitation: only works for **linearly separable** problems

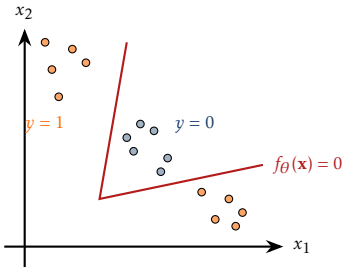
Solution: replace linear with neural network

$$\hat{y} = \mathbb{1}\left[f_\theta(\mathbf{x}) \geq 0\right]$$

Decision boundary:

Nonlinear surface $f_\theta(\mathbf{x}) = 0$





Biological vs Artificial Neuron

Biological neuron

- Dendrites receive signals
- Cell body integrates inputs
- Axon fires if threshold exceeded

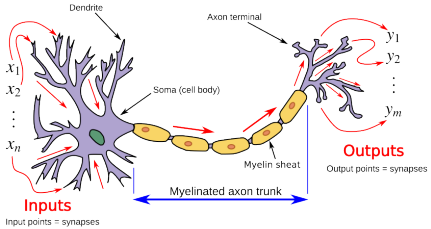
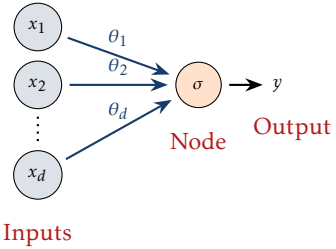


Image: Egm4313.s12 at English Wikipedia, CC BY-SA 3.0, via Wikimedia Commons

Artificial neuron $f : \mathbb{R}^d \to \mathbb{R}$

- Inputs: $\mathbf{x} \in \mathbb{R}^d$
- Activations: $\sigma(\boldsymbol{\theta}^\top \mathbf{x} + \theta_0)$
- Output: $y = f_\theta(\mathbf{x})$